```
# Import Necessary Libraries

import pandas as pd
import numpy as np

#Load the dataset

df = pd.read_csv("/content/dataset.csv", index_col = 0)
df.info()

# Check dataset dimensions
df.shape

# Check first few columns

df.head(5)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 114000 entries, 0 to 113999
Data columns (total 20 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   track_id          114000 non-null  object
 1   artists           113999 non-null  object
 2   album_name        113999 non-null  object
 3   track_name        113999 non-null  object
 4   popularity        114000 non-null  int64
 5   duration_ms       114000 non-null  int64
 6   explicit          114000 non-null  bool
 7   danceability      114000 non-null  float64
 8   energy            114000 non-null  float64
 9   key               114000 non-null  int64
 10  loudness          114000 non-null  float64
 11  mode              114000 non-null  int64
 12  speechiness       114000 non-null  float64
 13  acousticness      114000 non-null  float64
 14  instrumentalness  114000 non-null  float64
 15  liveness          114000 non-null  float64
 16  valence           114000 non-null  float64
 17  tempo             114000 non-null  float64
 18  time_signature    114000 non-null  int64
 19  track_genre       114000 non-null  object
dtypes: bool(1), float64(9), int64(5), object(5)
memory usage: 17.5+ MB
```

	track_id	artists	album_name	track_name	pop
0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	

1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic
2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again
3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love
4	5vjLSffimiIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On

Double-click (or enter) to edit

8.0 Data Quality Assessment

8.1 Completeness

```
# Check missing values across all columns

print(df.isnull().sum())
print((df.isnull().sum()/ len(df) * 100).round(3)) # missing value
```

```
track_id            0
artists             1
album_name          1
track_name          1
popularity          0
duration_ms         0
explicit            0
danceability        0
energy              0
key                 0
loudness            0
mode                0
speechiness         0
acousticness        0
instrumentalness    0
liveness            0
valence             0
tempo               0
time_signature      0
track_genre         0
dtype: int64
track_id            0.000
artists             0.001
album_name          0.001
track_name          0.001
popularity          0.000
duration_ms         0.000
explicit            0.000
danceability        0.000
energy              0.000
key                 0.000
loudness            0.000
mode                0.000
speechiness         0.000
acousticness        0.000
instrumentalness    0.000
liveness            0.000
valence             0.000
tempo               0.000
time_signature      0.000
track_genre         0.000
dtype: float64
```

8.2 Accuracy

```
# Step 1: summary statistics to spot suspicious values
num_cols = ['popularity','danceability','energy','loudness','speech
            'acousticness','instrumentalness','liveness','valence',
(df[num_cols].describe().round(3))
```

	popularity	danceability	energy	loudness	speechiness	ac
count	114000.000	114000.000	114000.000	114000.000	114000.000	
mean	33.239	0.567	0.641	-8.259	0.085	
std	22.305	0.174	0.252	5.029	0.106	
min	0.000	0.000	0.000	-49.531	0.000	
25%	17.000	0.456	0.472	-10.013	0.036	
50%	35.000	0.580	0.685	-7.004	0.049	
75%	50.000	0.695	0.854	-5.003	0.084	
max	100.000	0.985	1.000	4.532	0.965	

```
# Step 2: follow up on suspicious mins/maxes flagged above
print("tempo == 0:",       (df['tempo'] == 0).sum())
print("duration_ms == 0:", (df['duration_ms'] == 0).sum())
print("energy == 0:",      (df['energy'] == 0).sum())
print("loudness > 0:",     (df['loudness'] > 0).sum())

# Step 3: object columns
obj_cols = ['track_id','artists','album_name','track_name','track_g
for col in obj_cols:
    print(f"{col} — Unique: {df[col].nunique()}, Blank: {(df[col].s

# Step 4: other checks
print(df['track_id'].str.len().value_counts())
print(df['time_signature'].value_counts().sort_index())
print(df['explicit'].value_counts())
```

```
tempo == 0: 157
duration_ms == 0: 1
energy == 0: 1
loudness > 0: 90
track_id — Unique: 89741, Blank: 0
artists — Unique: 31437, Blank: 0
album_name — Unique: 46589, Blank: 0
track_name — Unique: 73608, Blank: 0
track_genre — Unique: 114, Blank: 0
track_id
22    114000
Name: count, dtype: int64
time_signature
0       163
1       973
3      9195
4    101843
5      1826
Name: count, dtype: int64
explicit
False    104253
True       9747
Name: count, dtype: int64
```

8.3 Consistency

```
#check for duplicate rows
print("Duplicate Rows:", df.duplicated().sum())

#Check duplicated "Track_id"
print("Duplicate track_ids" ,int(df["track_id"].duplicated().sum()))

#inspect example of the same track apperaing accross multiple genre

duplicate_example = df[df["track_id"].duplicated (keep=False)][['tr
duplicate_example
```

```
Duplicate Rows: 450
Duplicate track_ids 24259
```

	track_id	track_name	artists	track_genre
15028	001APMDOl3qtx1526T11n1	Better	Pink Sweat$;Kirby	chill
103211	001APMDOl3qtx1526T11n1	Better	Pink Sweat$;Kirby	soul
85578	001YQlnDSduXd5LgBd66gT	El Tiempo Es Dinero - Remasterizado 2007	Soda Stereo	punk-rock

Next steps: Generate code with duplicate_example New interactive sheet

8.4 Validity

```
# Out-of-range check for bounded features
range_cols = ['danceability','energy','speechiness','acousticness',
              'instrumentalness','liveness','valence']
for col in range_cols:
    print(f"{col} out of [0,1]:", ((df[col] < 0) | (df[col] > 1)).s

# Other range checks
print("popularity out of [0,100]:", ((df['popularity'] < 0) | (df['
print("key out of [0,11]:",         ((df['key'] < 0) | (df['key'] >
print("mode out of [0,1]:",         ((df['mode'] < 0) | (df['mode']

# Suspicious values (invalid zero values)
print("tempo > 250 BPM:",      (df['tempo'] > 250).sum())


# Check Duration outliers
df['duration_min'] = df['duration_ms'] / 60000
print("duration > 15 min:",    (df['duration_min'] > 15).sum())
print("duration > 30 min:",    (df['duration_min'] > 30).sum())


# Check zero popualrity tracks

print((df["popularity"] == 0).sum())
```

```
danceability out of [0,1]: 0
energy out of [0,1]: 0
speechiness out of [0,1]: 0
acousticness out of [0,1]: 0
instrumentalness out of [0,1]: 0
liveness out of [0,1]: 0
valence out of [0,1]: 0
popularity out of [0,100]: 0
key out of [0,11]: 0
mode out of [0,1]: 0
tempo > 250 BPM: 0
duration > 15 min: 153
duration > 30 min: 36
16020
```

9.0 Initial Data Wrangling

```
# Step 1: remove duplicate rows

df = df.drop_duplicates()
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 113550 entries, 0 to 113999
Data columns (total 21 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   track_id          113550 non-null  object
 1   artists           113549 non-null  object
 2   album_name        113549 non-null  object
 3   track_name        113549 non-null  object
 4   popularity        113550 non-null  int64
 5   duration_ms       113550 non-null  int64
 6   explicit          113550 non-null  bool
 7   danceability      113550 non-null  float64
 8   energy            113550 non-null  float64
 9   key               113550 non-null  int64
 10  loudness          113550 non-null  float64
 11  mode              113550 non-null  int64
 12  speechiness       113550 non-null  float64
 13  acousticness      113550 non-null  float64
 14  instrumentalness  113550 non-null  float64
 15  liveness          113550 non-null  float64
 16  valence           113550 non-null  float64
 17  tempo             113550 non-null  float64
 18  time_signature    113550 non-null  int64
 19  track_genre       113550 non-null  object
 20  duration_min      113550 non-null  float64
dtypes: bool(1), float64(10), int64(5), object(5)
memory usage: 18.3+ MB
None
```

```
#Step 2: Handling missing values

df = df.dropna(subset=["artists", "album_name","track_name"])
print(df.isnull().sum()) # To check if the missing values are remov
```

```
track_id            0
artists             0
album_name          0
track_name          0
popularity          0
duration_ms         0
explicit            0
danceability        0
energy              0
key                 0
loudness            0
mode                0
speechiness         0
acousticness        0
instrumentalness    0
liveness            0
valence             0
tempo               0
time_signature      0
track_genre         0
duration_min        0
dtype: int64
```

```
# Step 3: Remove zero duration track
df = df[df["duration_ms"] > 0]
```

```
# Step 4: Impute zero tempo with median

median_tempo = df[df["tempo"] > 0]["tempo"]. median()

df["tempo"] = df["tempo"].replace(0,median_tempo) # replace zeros w
print(median_tempo)
```

```
122.028
```

```
# Step 5: Engineering feature variables

# Transform explicit boolean to binary for modelling
df["explicit_int"] = df["explicit"].astype(int)

# Assign tier to popularity variable for descriptive analysis
df["popularity_tier"] = pd.cut(df['popularity'],
  bins = [0, 20, 40, 60, 80, 100],
  labels = ["Very Low", "Low", "Medium", "High", "Very High"])
```

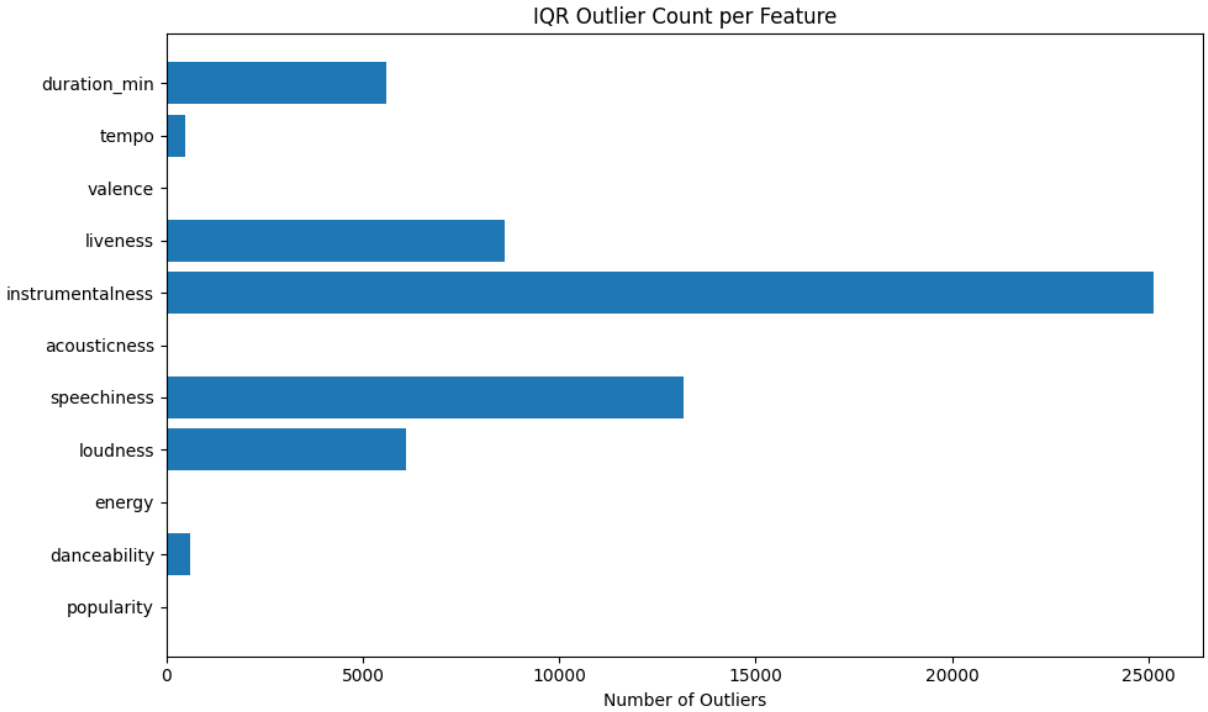
```
# Step 6: Standardise text formats

# Strip leading/trailing whitespace from all text columns
for col in ['artists', 'album_name', 'track_name', 'track_genre']:
    df[col] = df[col].str.strip()

# Standardise track_genre to lowercase for consistent matching
df['track_genre'] = df['track_genre'].str.lower()

# Verify no double spaces remain
print("Text standardisation:")
for col in ['artists', 'album_name', 'track_name']:
    double_space = df[col].str.contains(r'  ', na=False).sum()
    print(f"  {col} double spaces: {double_space}")
```

```
Text standardisation:
  artists double spaces: 0
  album_name double spaces: 0
  track_name double spaces: 0
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Step 7: Check outliers in the numerical columns

num_cols = ['popularity','danceability','energy','loudness','speech
            'acousticness','instrumentalness','liveness','valence',

outlier_counts = []

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = ((df[col] < lower) | (df[col] > upper)).sum()
```

```
    outlier_counts.append(outliers)
    print(f"{col} — outliers: {outliers}")  # ← indented inside loo

# Bar chart
plt.figure(figsize=(10, 6))
plt.barh(num_cols, outlier_counts)
plt.xlabel('Number of Outliers')
plt.title('IQR Outlier Count per Feature')
plt.tight_layout()
plt.savefig('outlier_barchart.png', dpi=150)
plt.show()
```

```
popularity — outliers: 2
danceability — outliers: 615
energy — outliers: 0
loudness — outliers: 6115
speechiness — outliers: 13167
acousticness — outliers: 0
instrumentalness — outliers: 25131
liveness — outliers: 8610
valence — outliers: 0
tempo — outliers: 479
duration_min — outliers: 5592
```



```
# Step 8: Normalisation (min-max scaling)

# Apply mix-max scaling to the numeric variables

scaling_cols = ['danceability', 'energy', 'loudness', 'speechiness'
              'instrumentalness', 'liveness', 'valence', 'tempo', '

for col in scaling_cols:
    col_min = df[col].min()
    col_max = df[col].max()
    df[col + '_norm'] = ((df[col] - col_min) / (col_max - col_min))

norm_cols = [c + '_norm' for c in scaling_cols]
print(df[norm_cols].agg(['min', 'max']).round(4))

# Sample view of comparision of before and after normalisation (lou

print(df[['track_name', 'loudness', 'loudness_norm']].head(10).to_s
```

```
     danceability_norm  energy_norm  loudness_norm  speechiness_norm
min                0.0          0.0            0.0               0.0
max                1.0          1.0            1.0               1.0

     acousticness_norm  instrumentalness_norm  liveness_norm  valenc
min                0.0                    0.0            0.0
max                1.0                    1.0            1.0

     tempo_norm  duration_min_norm
min         0.0                0.0
max         1.0                1.0
                   track_name  loudness  loudness_norm
0                      Comedy    -6.746         0.7914
1             Ghost - Acoustic   -17.235         0.5974
2               To Begin Again    -9.734         0.7361
3  Can't Help Falling In Love   -18.515         0.5737
4                     Hold On    -9.681         0.7371
5        Days I Will Remember    -8.807         0.7533
6               Say Something    -8.822         0.7530
7                   I'm Yours    -9.331         0.7436
8                       Lucky    -8.700         0.7552
9                      Hunger    -6.770         0.7909
```

```
# Step 10: Final dataset verification

print("Final shape:       ", df.shape)
print("Missing values:    ", df.isnull().sum().sum())
print("Duplicate rows:    ", df.duplicated().sum())
print("Zero tempo:        ", (df['tempo'] == 0).sum())
print("Zero duration_ms:  ", (df['duration_ms'] == 0).sum())
print("Original columns:  ", 20)
print("New columns added: ", df.shape[1] - 20)
```

```
Final shape:        (113549, 33)
Missing values:     15843
Duplicate rows:     0
Zero tempo:         0
Zero duration_ms:   0
Original columns:   20
New columns added:  13
```

```
# Save cleaned dataset

df.to_csv("dataset_clean_final.csv", index=False)
print("\nCleaned dataset saved as dataset_clean_final.csv")
```

```
Cleaned dataset saved as dataset_clean_final.csv
```

10.0 Exploratory Data Analysis

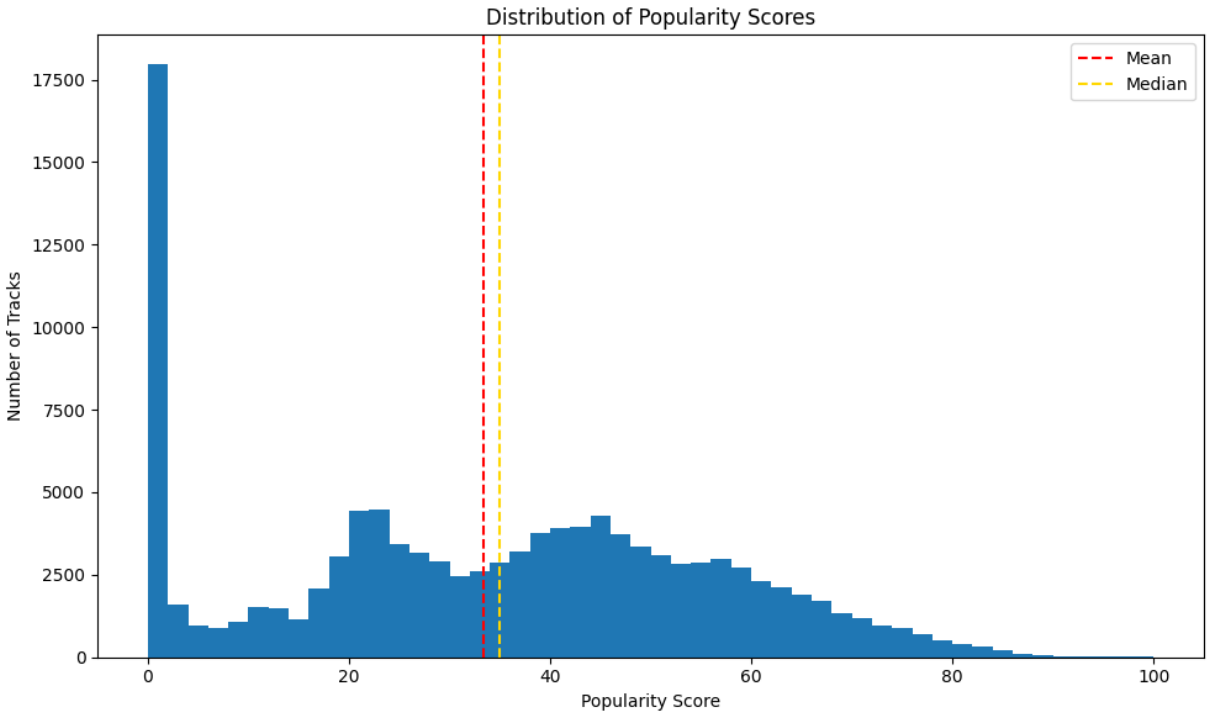
10.1 Univariate Analysis

10.1.1 Popularity Distribution

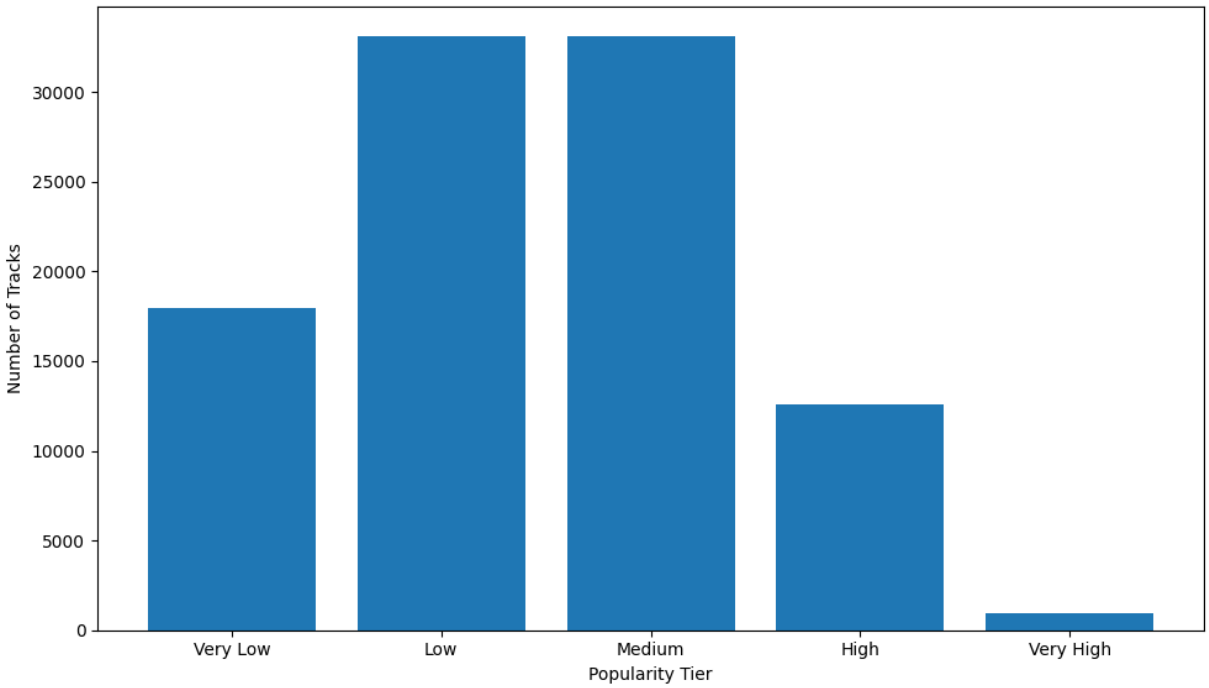
```
# Target Variable - Popularity

#Populrity Skewness and kurtosis

print("Skewness:", round(df['popularity'].skew(), 3))
print("Kurtosis:", round(df['popularity'].kurtosis(),3))
print("Mean:", round(df['popularity'].mean(),3))
print("Median:", round(df['popularity'].median(),3))


# Histogram of popularity scores
```

```
plt.figure(figsize=(10, 6))
plt.hist(df['popularity'], bins=50)
plt.axvline(df['popularity'].mean(), color='red', linestyle='--', l
plt.axvline(df['popularity'].median(), color='gold', linestyle='--'
plt.title("Distribution of Popularity Scores")
plt.xlabel("Popularity Score")
plt.ylabel("Number of Tracks")
plt.legend()
plt.tight_layout()
plt.savefig('popularity_histogram.png', dpi=150)
plt.show()


# Popularity tier bar chart

Tier_counts = df['popularity_tier'].value_counts().sort_index()
plt.figure(figsize=(10, 6))
plt.bar(Tier_counts.index, Tier_counts.values)
plt.xlabel('Popularity Tier')
plt.ylabel('Number of Tracks')
plt.title('Popularity Tier Distribution')
plt.tight_layout()
plt.savefig('popularity_tier_barchart.png', dpi=150)
plt.show()
```

```
Skewness: 0.042
Kurtosis: -0.924
Mean: 33.324
Median: 35.0
```



Popularity Tier Distribution

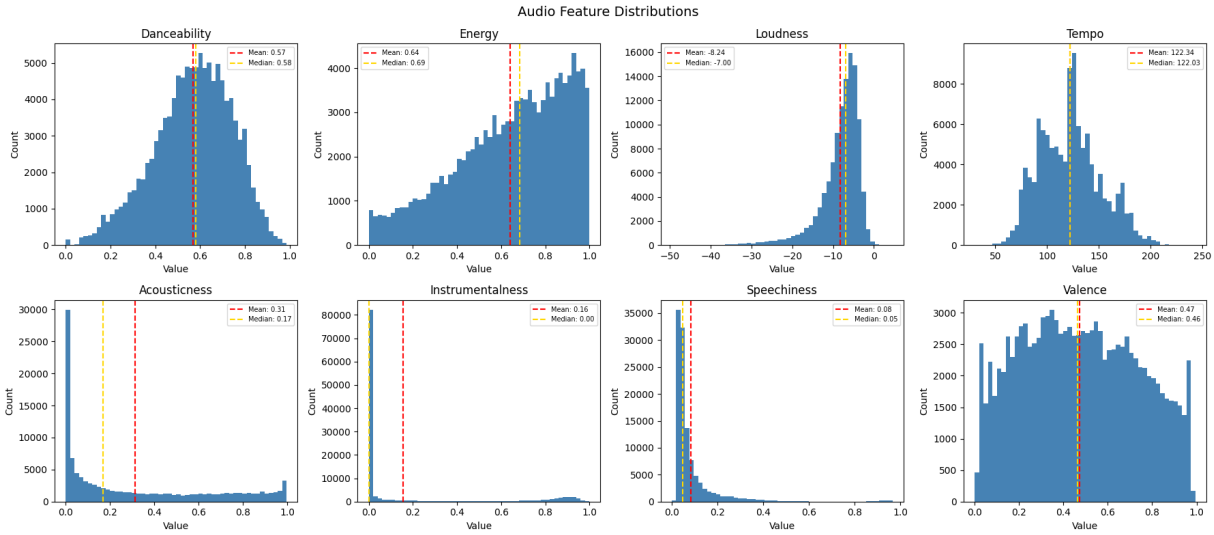


10.1.2 Audio Feature Distribution

```
# Audio features distribution

all_audio_features = ["danceability", "energy","loudness", "tempo",
                      "acousticness", "instrumentalness", "speechin
```

```
# Skewness of each audio feature

print((round(df[all_audio_features].skew(),3)))

# Histograms for each auido feature (to be shown together for bette

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
axes = axes.flatten()

for i, col in enumerate(all_audio_features):
    axes[i].hist(df[col], bins=50, color='steelblue')
    axes[i].axvline(df[col].mean(), color='red', linestyle='--', la
    axes[i].axvline(df[col].median(), color='gold', linestyle='--',
    axes[i].set_title(col.capitalize())
    axes[i].set_xlabel('Value')
    axes[i].set_ylabel('Count')
    axes[i].legend(fontsize=7)

plt.suptitle('Audio Feature Distributions', fontsize=14)
plt.tight_layout()
plt.savefig('eda_audio_features_grid.png', dpi=150)
plt.show()
```

```
danceability       -0.400
energy             -0.599
loudness           -2.013
tempo               0.320
acousticness        0.730
instrumentalness    1.738
speechiness         4.645
valence             0.115
dtype: float64
```

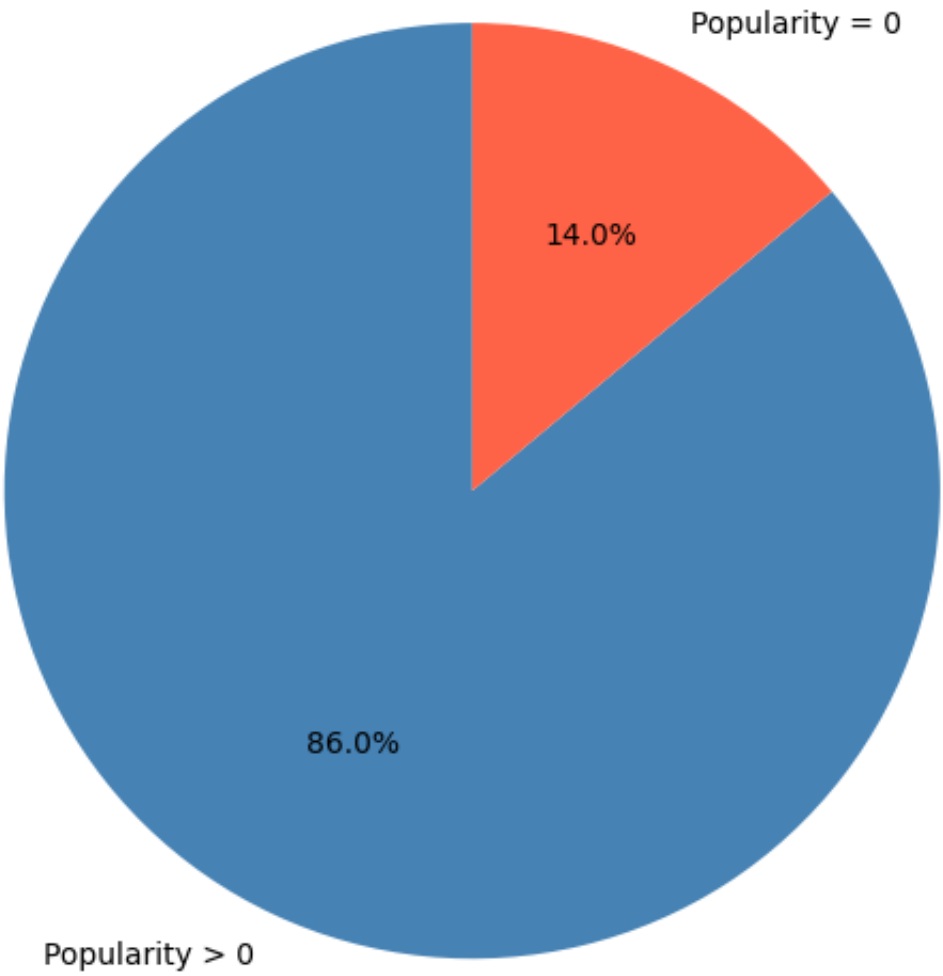


10.1.3 Zero Popularity Distribution

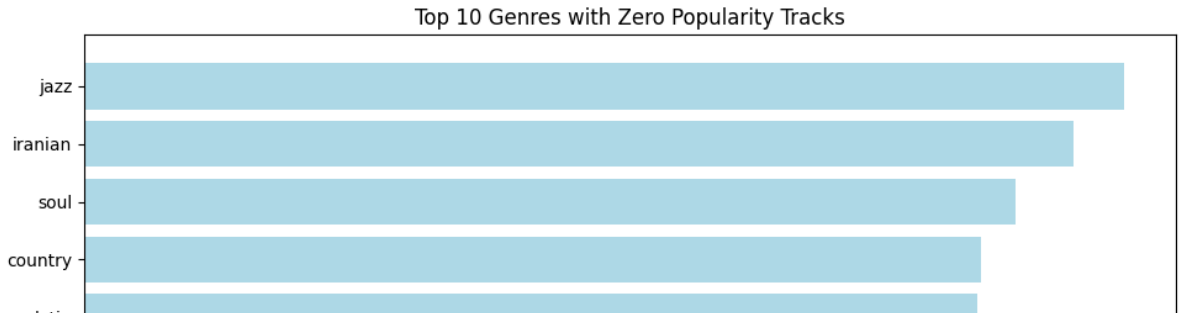
```
# Zero Popularity distribution

# Count tracks with zero popularity

zero_pop = (df['popularity'] == 0).sum()
print("Number of tracks with zero popularity:", zero_pop) # 177 tra

# Pie chart showing distribution between zero and non-zero populari
plt.figure(figsize=(6, 6))
plt.pie([len(df) - zero_pop, zero_pop],
labels=['Popularity > 0', 'Popularity = 0'],
        autopct='%1.1f%%',
        colors=['steelblue', 'tomato'],
        startangle=90)
plt.title ("Zero Vs Non-Zero Popularity Tracks")
plt.tight_layout()
plt.savefig('zero_popularity_piechart.png', dpi=150)
plt.show()


# Top genres with zero popularity

zero_pop_genres = df[df['popularity'] == 0]['track_genre'].value_co
print(zero_pop_genres)

# Barchart visualising the information for better interpretation

plt.figure(figsize=(10, 6))
plt.barh(zero_pop_genres.index[::-1], zero_pop_genres.values[::-1],
plt.title('Top 10 Genres with Zero Popularity Tracks')
plt.xlabel('Number of Zero-Popularity Tracks')
plt.tight_layout()
plt.savefig('eda_zero_genre.png', dpi=150)
plt.show()
```

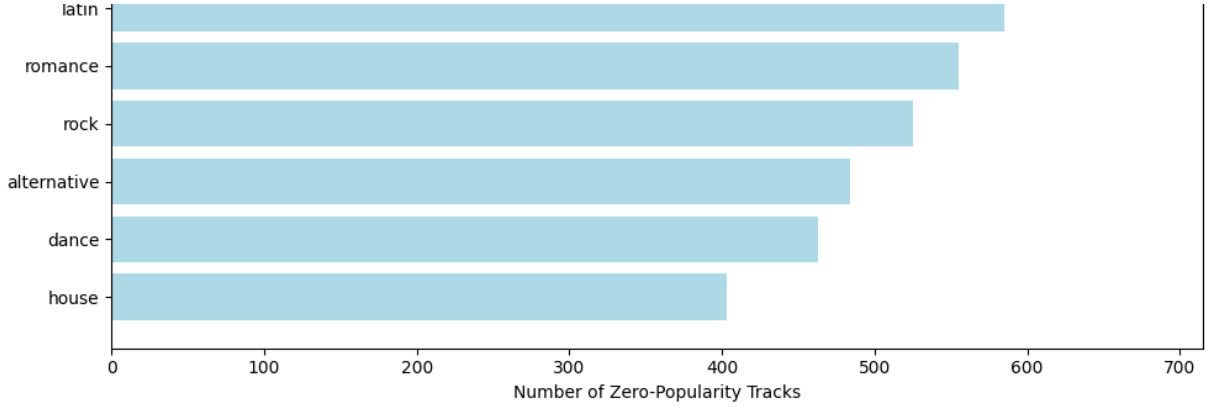
```
Number of tracks with zero popularity: 15843
```

Zero Vs Non-Zero Popularity Tracks




```
track_genre
jazz            681
iranian         648
soul            610
country         587
latin           585
romance         555
rock            525
alternative     484
dance           463
house           403
Name: count, dtype: int64
```



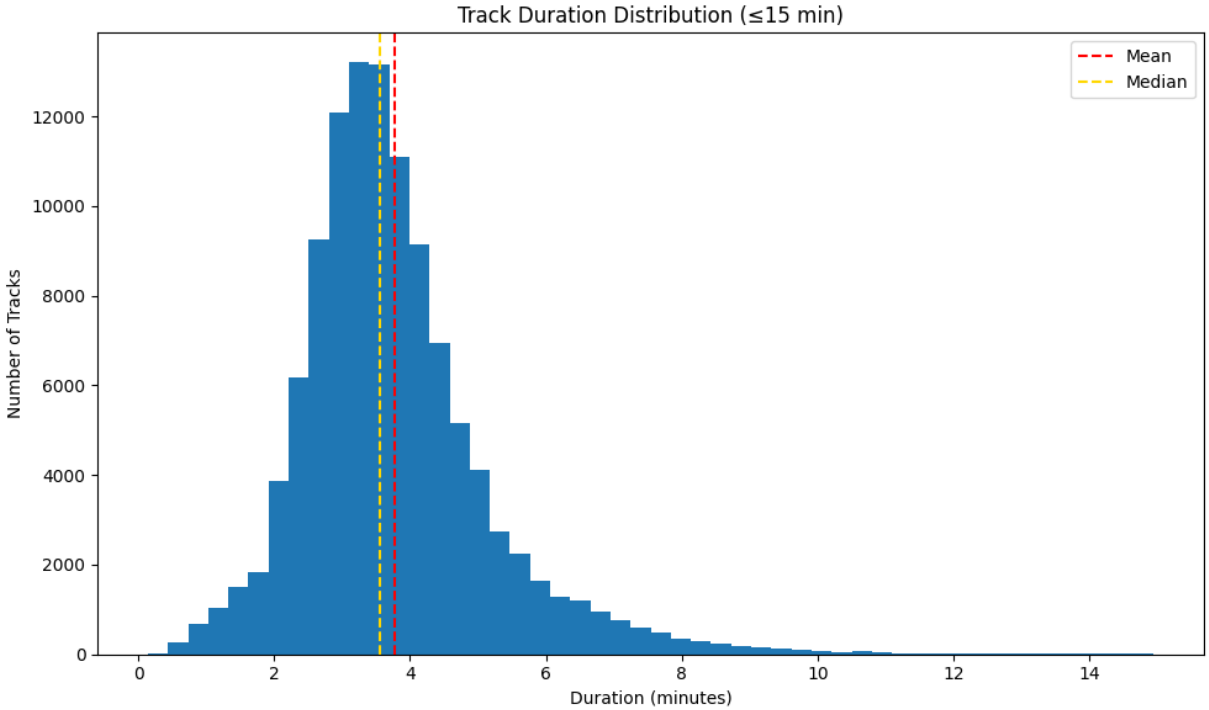


10.1.4 Track Duration Distribution

```
# Generate summary statistics of track duration

print(df["duration_min"].describe().round(2))

# Historgram showing track duration distribution

df_dur = df[df['duration_min'] <= 15]



plt.figure(figsize=(10, 6))
plt.hist(df_dur['duration_min'], bins=50)
plt.axvline(df_dur['duration_min'].mean(), color='red', linestyle='
plt.axvline(df_dur['duration_min'].median(), color='gold', linestyl
plt.title('Track Duration Distribution (≤15 min)')
plt.xlabel('Duration (minutes)')
plt.ylabel('Number of Tracks')
plt.legend()
plt.tight_layout()
plt.savefig('eda_duration.png', dpi=150)
plt.show()
```

```
print("duration > 15 min:",    (df['duration_min'] > 15).sum())
```

```
count    113549.00
mean          3.80
std           1.77
min           0.14
25%           2.90
50%           3.55
75%           4.36
max          87.29
Name: duration_min, dtype: float64
```



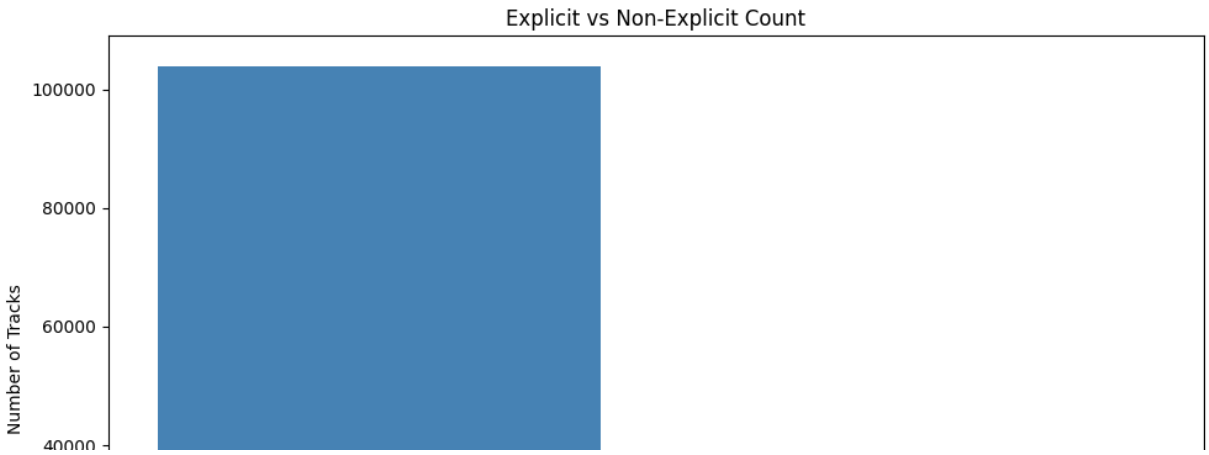

```
duration > 15 min: 151
```

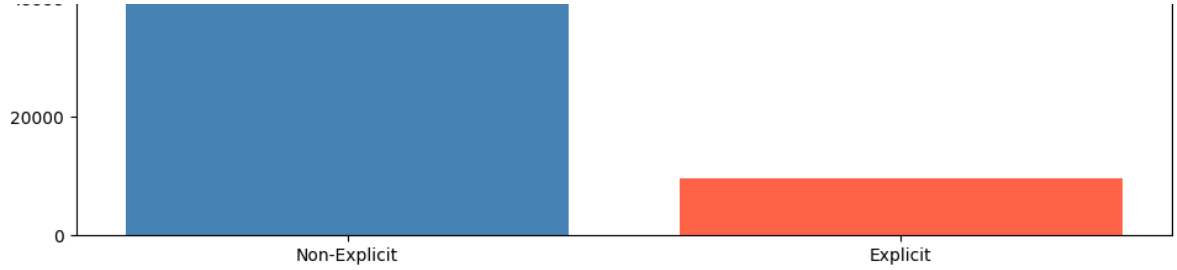
10.1.5 Explicit Content Distribution

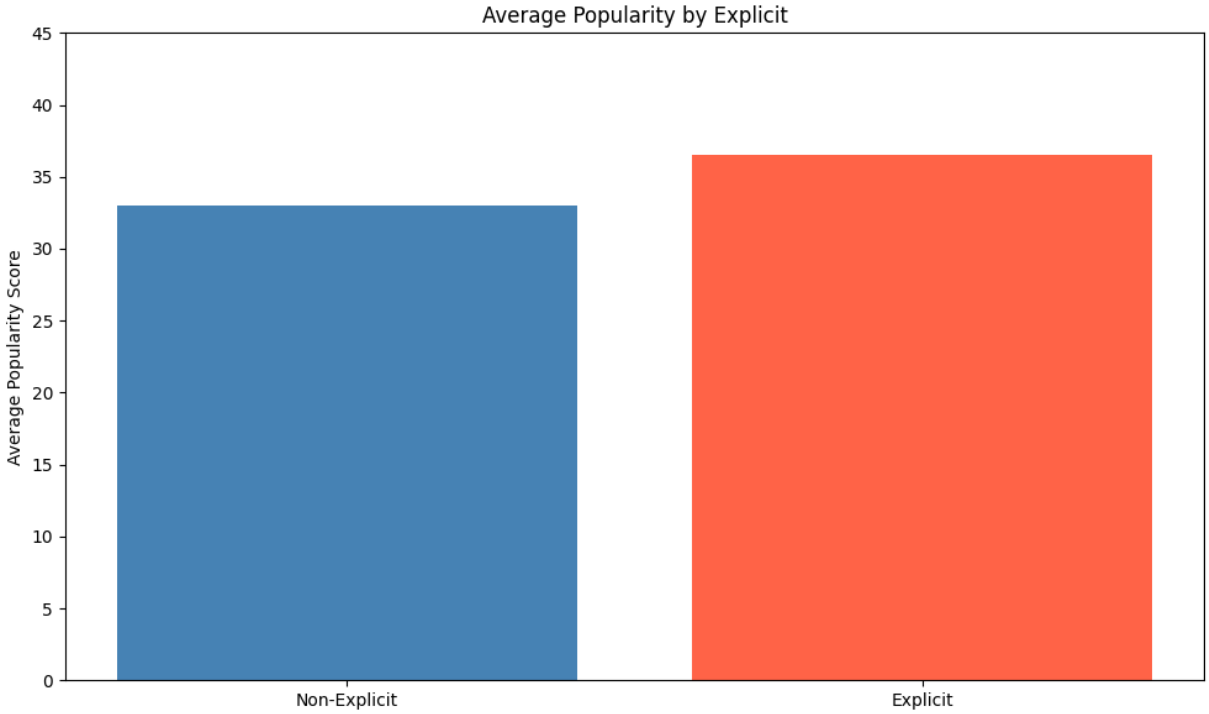
```
# Explicit summary statisitics
```

```
ex_count = df["explicit"].value_counts()
ex_pop = df.groupby('explicit')['popularity'].mean().round(2)

print(ex_count)
print(ex_pop)


# Bar chart showing explicit and non-explicit count

plt.figure(figsize=(10, 6))
plt.bar(['Non-Explicit', 'Explicit'], ex_count.values, color=['stee
plt.title('Explicit vs Non-Explicit Count')
plt.ylabel('Number of Tracks')
plt.tight_layout()
plt.savefig('eda_explicit_count.png', dpi=150)
plt.show()

# Bar chart showing average popularity by explicit

plt.figure(figsize=(10, 6))
plt.bar(['Non-Explicit', 'Explicit'], ex_pop.values, color=['steelb
plt.title('Average Popularity by Explicit')
plt.ylabel('Average Popularity Score')
plt.ylim(0, 45)
plt.tight_layout()
plt.savefig('eda_explicit_pop.png', dpi=150)
plt.show()
```

```
explicit
False    103831
True       9718
Name: count, dtype: int64
explicit
False    33.02
True     36.52
Name: popularity, dtype: float64
```




20000
0
Non-Explicit
Explicit

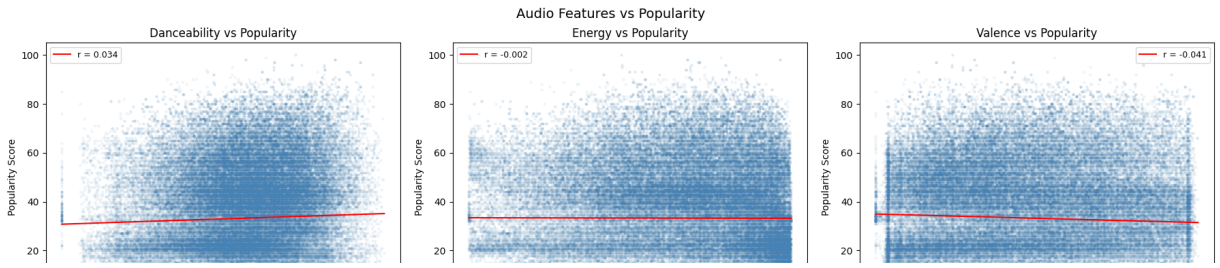

Average Popularity by Explicit
Average Popularity Score
45
40
35
30
25
20
15
10
5
0
Non-Explicit
Explicit

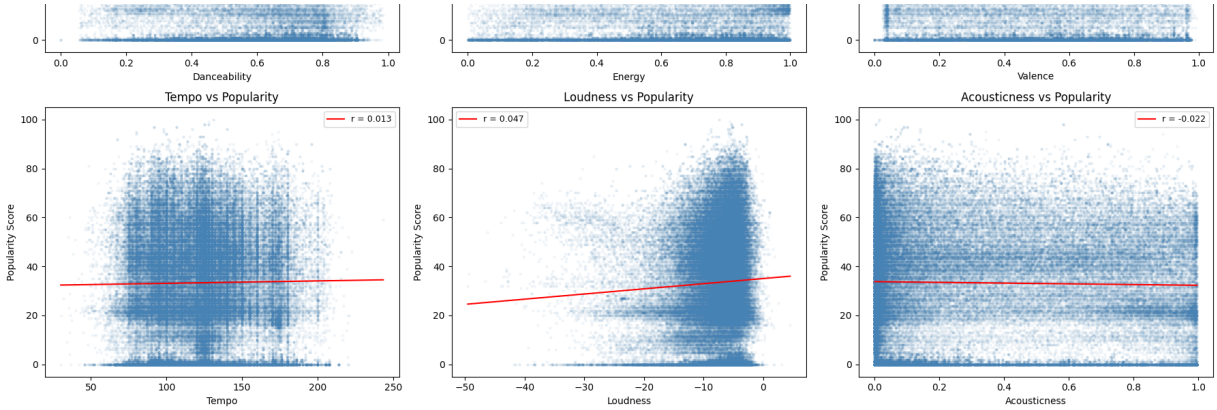
10.2 Bivariate Analysis

10.2.1 Which Audio Features Covary Most Strongly with Popularity?

10.2.2 How Does Each Audio Feature Individually Correlate to Popularity?

```
# Chosen Top features based on pearson correlation

features = ['danceability', 'energy', 'valence', 'tempo', 'loudness

# Generate scatter plot

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for i, col in enumerate(features):
    axes[i].scatter(df[col], df['popularity'], alpha=0.05, color='s
    m, b = np.polyfit(df[col], df['popularity'], 1)
    x_line = np.linspace(df[col].min(), df[col].max(), 100)
    axes[i].plot(x_line, m * x_line + b, color='red', linewidth=1.5
                 label=f'r = {round(df[col].corr(df["popularity"]),
    axes[i].set_title(f'{col.capitalize()} vs Popularity')
    axes[i].set_xlabel(col.capitalize())
    axes[i].set_ylabel('Popularity Score')
    axes[i].legend(fontsize=9)

plt.suptitle('Audio Features vs Popularity', fontsize=14)
plt.tight_layout()
plt.savefig('scatter_grid.png', dpi=150)
plt.show()

# Get values for slope
for col in features:
    r = round(df[col].corr(df['popularity']), 3)
    m, b = np.polyfit(df[col], df['popularity'], 1)
    print(f"slope = {round(m, 3)}")
```

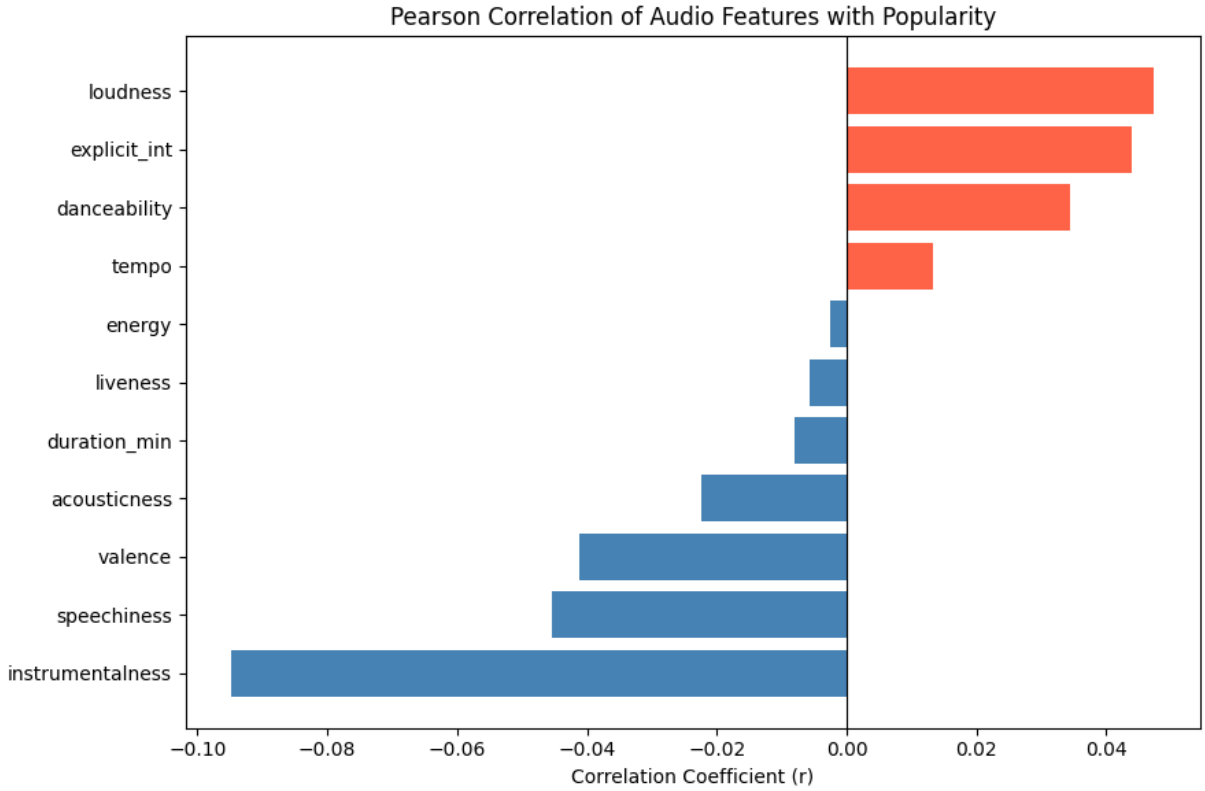





```
slope = 4.421
slope = -0.217
slope = -3.533
slope = 0.01
slope = 0.211
slope = -1.501
```

```
# Creating a new variable with just the variables for correlation

corr_cols = ['danceability','energy','loudness','speechiness','acou
             'instrumentalness','liveness','valence','tempo','durat

# Calculate correlation of each feature with popularity
cor = df[corr_cols + ['popularity']].corr()['popularity'].drop('pop
print(cor.round(3))

# Generate bar chart visualising the correalation

plt.figure(figsize=(9, 6))
colors = ['steelblue' if val < 0 else 'tomato' for val in cor.value
plt.barh(cor.index, cor.values, color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Pearson Correlation of Audio Features with Popularity')
plt.xlabel('Correlation Coefficient (r)')
plt.tight_layout()
plt.savefig('eda_correlation.png', dpi=150)
plt.show()
```

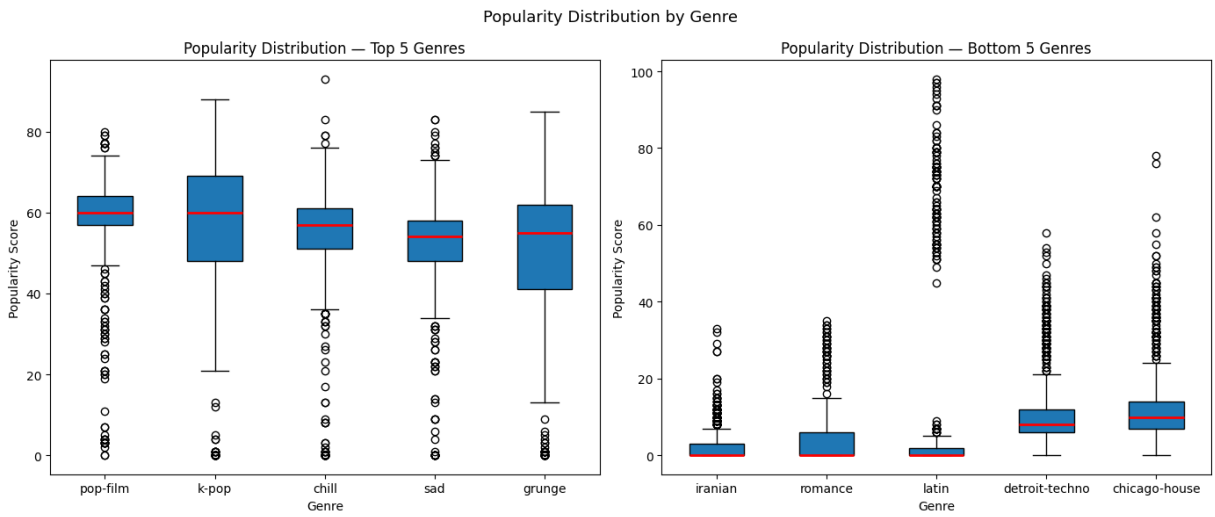
```
instrumentalness   -0.095
speechiness        -0.045
valence            -0.041
acousticness       -0.022
duration_min       -0.008
liveness           -0.006
energy             -0.002
tempo               0.013
danceability        0.034
explicit_int        0.044
loudness            0.047
Name: popularity, dtype: float64
```



10.2.3 Does Genre Significantly Affect Popularity Distribution?

```
top5_genres = df.groupby('track_genre')['popularity'].mean().sort_v
```

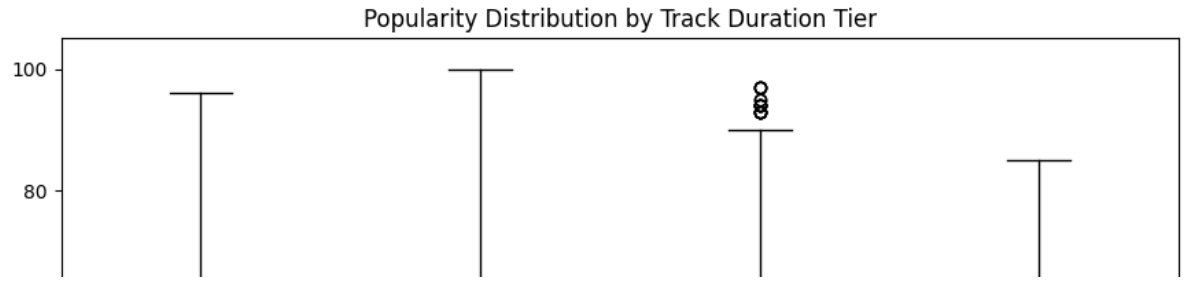
```
bot5_genres  = df.groupby('track_genre')['popularity'].mean().sort_

top5_data = [df[df['track_genre'] == g]['popularity'].values for g
bot5_data  = [df[df['track_genre'] == g]['popularity'].values for g

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].boxplot(top5_data, tick_labels=top5_genres, patch_artist=Tr
                medianprops=dict(color='red', linewidth=2))
axes[0].set_title('Popularity Distribution — Top 5 Genres')
axes[0].set_ylabel('Popularity Score')
axes[0].set_xlabel('Genre')

axes[1].boxplot(bot5_data, tick_labels=bot5_genres, patch_artist=Tr
                medianprops=dict(color='red', linewidth=2))
axes[1].set_title('Popularity Distribution — Bottom 5 Genres')
axes[1].set_ylabel('Popularity Score')
axes[1].set_xlabel('Genre')

plt.suptitle('Popularity Distribution by Genre', fontsize=13)
plt.tight_layout()
plt.savefig('eda_genre_boxplot.png', dpi=150)
plt.show()
```

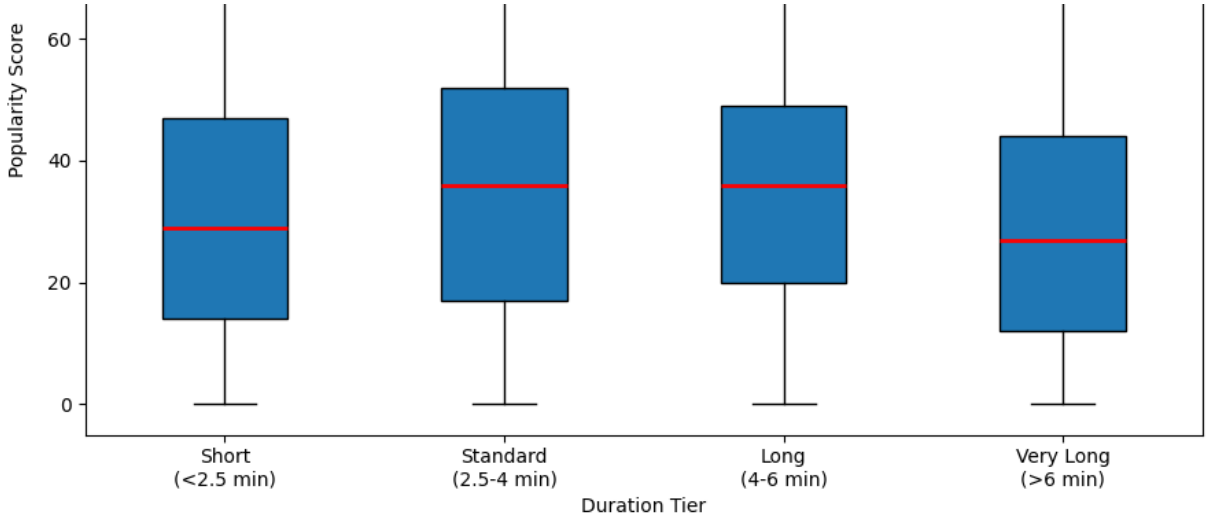


10.2.4 Do Tracks of Standard Commerical Length Perform Better?

```
# Create duration tiers
df['duration_tier'] = pd.cut(df['duration_min'],
    bins=[0, 2.5, 4, 6, 100],
    labels=['Short (<2.5)', 'Standard (2.5-4)', 'Long (4-6)', 'Very

# Summary statistics by tier
print("Popularity by duration tier:")
print(df.groupby('duration_tier')['popularity'].agg(['mean', 'media

# Group popularity values by tier for box plot
short    = df[df['duration_tier'] == 'Short (<2.5)']['popularity'].
standard = df[df['duration_tier'] == 'Standard (2.5-4)']['popularit
long     = df[df['duration_tier'] == 'Long (4-6)']['popularity'].va
very_long  = df[df['duration_tier'] == 'Very Long (>6)']['popularit

# Generate Box plot
plt.figure(figsize=(9, 6))
plt.boxplot([short, standard, long, very_long],
            tick_labels=['Short\n(<2.5 min)', 'Standard\n(2.5-4 min
                         'Long\n(4-6 min)', 'Very Long\n(>6 min)'],
            patch_artist=True,
            medianprops=dict(color='red', linewidth=2))
plt.title('Popularity Distribution by Track Duration Tier')
plt.ylabel('Popularity Score')
plt.xlabel('Duration Tier')
plt.tight_layout()
plt.savefig('eda_duration_boxplot.png', dpi=150)
plt.show()
```

```
Popularity by duration tier:
                   mean  median  count
duration_tier
Short (<2.5)      30.56    29.0  15100
Standard (2.5-4)  34.06    36.0  59386
Long (4-6)        34.34    36.0  31341
Very Long (>6)    28.91    27.0   7722
/tmp/ipykernel_6910/3423926151.py:8: FutureWarning: The default of o
  print(df.groupby('duration_tier')['popularity'].agg(['mean', 'medi
```



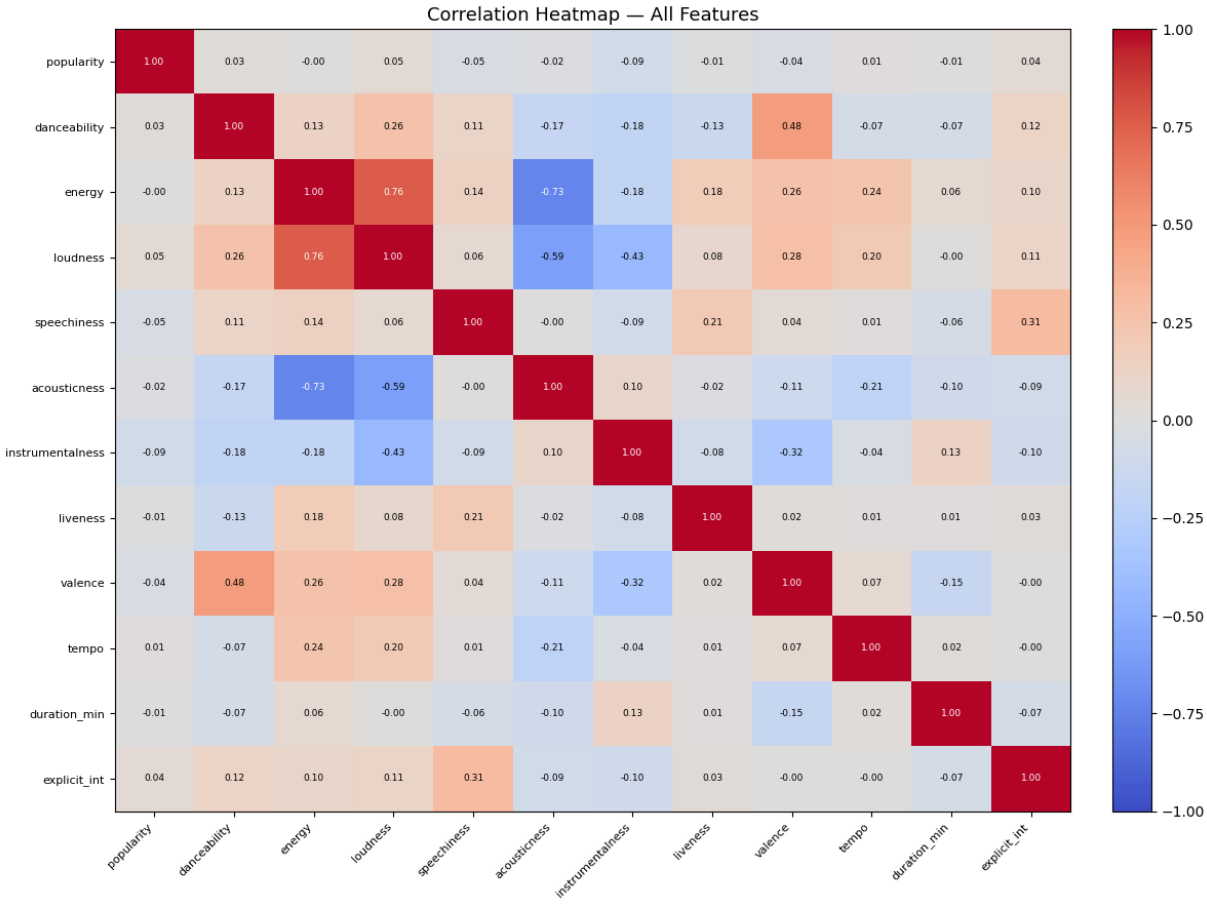


10.3 Multivariate Analysis

10.3.1 How Do All Features Correlate Simultaneously?

```
# Generate correlation heat map

corr_cols = ['popularity','danceability','energy','loudness','speech
             'acousticness','instrumentalness','liveness','valence',
             'duration_min','explicit_int']

corr_matrix = df[corr_cols].corr().round(2)

plt.figure(figsize=(12, 9))
im = plt.imshow(corr_matrix.values, cmap='coolwarm', vmin=-1, vmax=1
plt.colorbar(im, fraction=0.046, pad=0.04)
plt.xticks(range(len(corr_cols)), corr_cols, rotation=45, ha='right'
plt.yticks(range(len(corr_cols)), corr_cols, fontsize=8)
for i in range(len(corr_cols)):
    for j in range(len(corr_cols)):
```

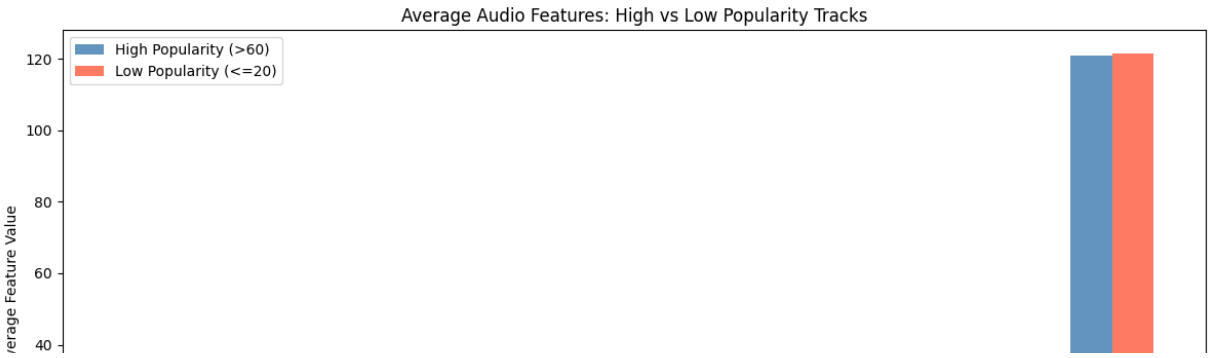
```
    for j in range(len(corr_cols)):
        val = corr_matrix.values[i, j]
        color = 'white' if abs(val) > 0.6 else 'black'
        plt.text(j, i, f'{val:.2f}', ha='center', va='center', fonts
plt.title('Correlation Heatmap — All Features', fontsize=13)
plt.tight_layout()
plt.savefig('eda_heatmap.png', dpi=150)
plt.show()
```

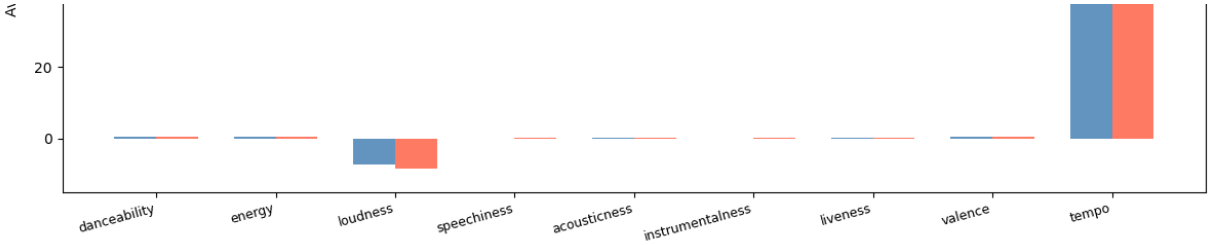


10.3.2 What Differntiates High Popularity Tracks from Low Popularity Tracks Across Multiple Features?

```
# Split into high and low popularity groups
high = df[df['popularity'] > 60]
low  = df[df['popularity'] <= 20]

audio_cols = ['danceability','energy','loudness','speechiness',
              'acousticness','instrumentalness','liveness','valence

high_means = high[audio_cols].mean().round(3)
low_means  = low[audio_cols].mean().round(3)

print(pd.DataFrame({'High Pop (>60)': high_means, 'Low Pop (<=20)':

x = np.arange(len(audio_cols))
w = 0.35

plt.figure(figsize=(12, 6))
plt.bar(x - w/2, high_means.values, w, label='High Popularity (>60)
plt.bar(x + w/2, low_means.values,  w, label='Low Popularity (<=20)
plt.xticks(x, audio_cols, rotation=15, ha='right', fontsize=9)
plt.title('Average Audio Features: High vs Low Popularity Tracks')
plt.ylabel('Average Feature Value')
plt.legend()
plt.tight_layout()
plt.savefig('eda_high_vs_low.png', dpi=150)
plt.show()
```

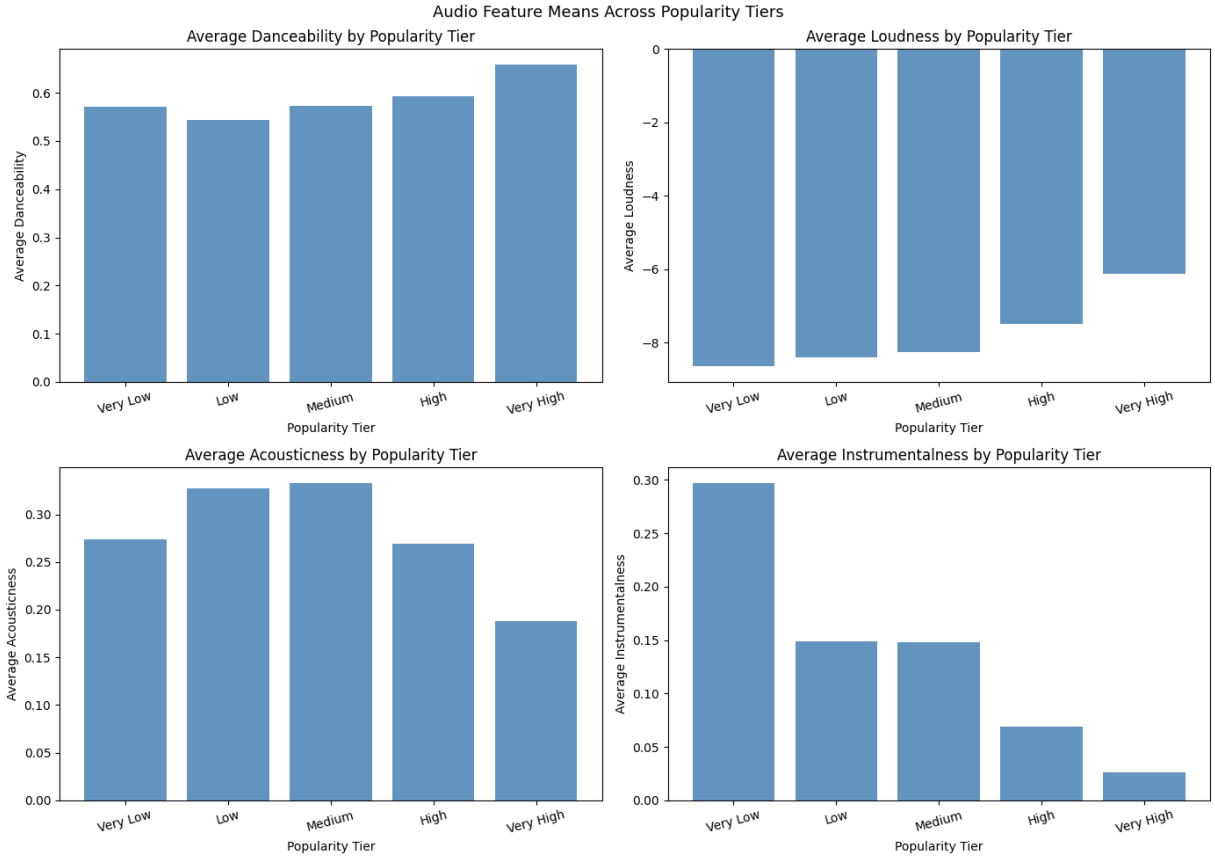
```
                  High Pop (>60)  Low Pop (<=20)
danceability               0.597           0.573
energy                     0.649           0.640
loudness                  -7.386          -8.425
speechiness                0.077           0.085
acousticness               0.264           0.304
instrumentalness           0.066           0.206
liveness                   0.180           0.194
valence                    0.486           0.483
tempo                    120.834         121.481
```





10.3.3 How do All Audio Feature Change Across Popularity Tiers?

```
# Define popularity tier order and key features to analyse
tier_order   = ['Very Low', 'Low', 'Medium', 'High', 'Very High']
key_features = ['danceability', 'loudness', 'acousticness', 'instrum

# Print average feature values per popularity tier
print(df.groupby('popularity_tier')[key_features].mean().round(3).re

# Create 2x2 grid of bar charts — one per feature
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(key_features):
    # Calculate mean feature value for each popularity tier
    tier_means = df.groupby('popularity_tier')[col].mean().reindex(t

    # Plot bar chart
    axes[i].bar(tier_means.index, tier_means.values, color='steelblu
    axes[i].set_title(f'Average {col.capitalize()} by Popularity Tie
    axes[i].set_xlabel('Popularity Tier')
    axes[i].set_ylabel(f'Average {col.capitalize()}')
    axes[i].tick_params(axis='x', rotation=15)

plt.suptitle('Audio Feature Means Across Popularity Tiers', fontsize=
plt.tight_layout()
plt.savefig('eda_tier_features.png', dpi=150)
plt.show()
```

```
/tmp/ipykernel_6910/468894537.py:6: FutureWarning: The default of ob
```

```
  print(df.groupby('popularity_tier')[key_features].mean().round(3).
/tmp/ipykernel_6910/468894537.py:14: FutureWarning: The default of o
  tier_means = df.groupby('popularity_tier')[col].mean().reindex(tie
/tmp/ipykernel_6910/468894537.py:14: FutureWarning: The default of o
  tier_means = df.groupby('popularity_tier')[col].mean().reindex(tie
/tmp/ipykernel_6910/468894537.py:14: FutureWarning: The default of o
  tier_means = df.groupby('popularity_tier')[col].mean().reindex(tie
/tmp/ipykernel_6910/468894537.py:14: FutureWarning: The default of o
  tier_means = df.groupby('popularity_tier')[col].mean().reindex(tie
                danceability  loudness  acousticness  instrumentaln
popularity_tier
Very Low               0.572    -8.637         0.274             0.
Low                    0.543    -8.399         0.327             0.
Medium                 0.572    -8.255         0.332             0.
High                   0.593    -7.481         0.269             0.
Very High              0.658    -6.128         0.188             0.
```